

Simulação de uma CPU baseada na Arquitetura de Von Neumann em Python

Implementação prática e lógica de execução

Samuel Valentim e Pedro Cunha

08 de abril de 2026

1 - INTRODUÇÃO

A arquitetura de computadores é a área da computação que estuda como funcionam os sistemas computacionais, assim como sua estrutura interna e a organização dos componentes. Essa área tem como foco entender como os computadores processam dados e executam instruções, além de definir o modo em que o hardware funciona e se comunica com o software.

Com o passar dos anos e a demanda crescendo cada vez mais, a arquitetura de computadores se tornou essencial para o desenvolvimento de sistemas que fizessem jus a demanda, ou seja, serem cada vez mais rápidos, seguros e eficientes. Desse modo, a organização dos componentes do computador, como o processador, dispositivos Input e Output e a memória, influenciam diretamente no desempenho da máquina, como vemos atualmente a importância dessa tecnologia no nosso cotidiano.

Entre os modelos mais conhecidos e importantes dessa área, podemos destacar a arquitetura de Von Neumann, que se tornou fundamental para a maioria dos computadores modernos. Sua estrutura é composta por componentes fundamentais como a Unidade Central de Processamento (CPU), a Memória Principal (RAM), os Dispositivos de Entrada ou Saída (Input/Output) e os Barramentos responsáveis pela comunicação entre esses componentes.

Portanto, o trabalho em questão tem como objetivo estudar a arquitetura de Von Neumann e seus componentes e, através deles, desenvolver uma simulação de uma CPU baseada nesse modelo, utilizando a linguagem de programação de alto nível Python, dessa forma então, compreendendo na prática o funcionamento interno de um sistema computacional.

2 - DESENVOLVIMENTO DO SISTEMA

Nessa seção iremos apresentar os principais componentes da Arquitetura de Von Neumann, explicando seu funcionamento e a importância do mesmo em um sistema computacional.

2.1 - Componentes da Arquitetura de Von Neumann

Essa arquitetura é a base para o funcionamento da maioria dos computadores da nossa atualidade. Nesse projeto, iremos focar nos 5 componentes principais da arquitetura de Neumann.

2.1.1 - CPU (Unidade Central de Processamento)

A CPU é o componente principal, responsável por executar as instruções. Além disso, também é composta pela **ULA** (Unidade Lógica e Aritmética), que realiza os cálculos matemáticos e os testes lógicos e a **Unidade de Controle (UC)** que controla o sistema.

2.1.2 - Memória Principal (RAM)

A memória RAM é um componente volátil e de rápido acesso que armazena dados temporários e instruções usadas pelo computador. Na arquitetura de Von Neumann, não há separação física entre o local que armazena um **número** (dado) e o local onde se guarda um **comando** (instrução).

2.1.3 - Barramentos

Os Barramentos funcionam como caminhos onde ocorre a comunicação da **CPU à memória** e os **dispositivos I/O** permitindo o tráfego de informações.

2.1.4 - Dispositivos de Entrada e Saída (I/O)

Os I/O 's são os mecanismos que permitem os nossos computadores interagirem com o mundo externo, recebendo novos dados que podem ser processados ou entregar resultados obtidos.

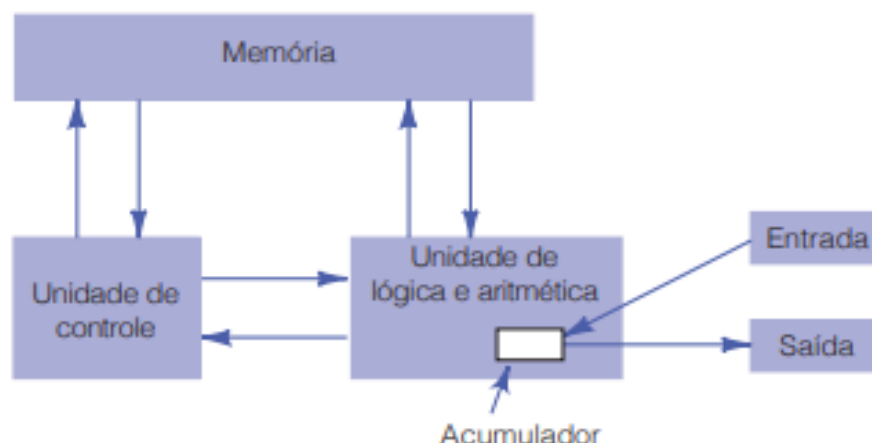
2.1.5 - Ciclo de Instrução

O Ciclo de Instrução é o que define o funcionamento em sequência da máquina, podendo ser dividido em três etapas, sendo elas: **Busca** (Fetch) da instrução na memória, **Decodificação** (Decode) do comando e a **Execução** (Execute) da operação.

2.1.6 - Conclusão do Tópico

A estrutura da Arquitetura de Von Neumann possibilita que a mesma máquina possa executar diferentes tarefas com uma simples mudança, na qual o software é alterado na memória, de tal forma que esse princípio fundamenta os computadores modernos e será o guia para a implementação em Python nesse projeto prático

Figura 1 - Arquitetura da máquina original de Von Neumann.



Fonte: Tanenbaum e Austin (2013).

2.2 - Implementação do simulador em Python

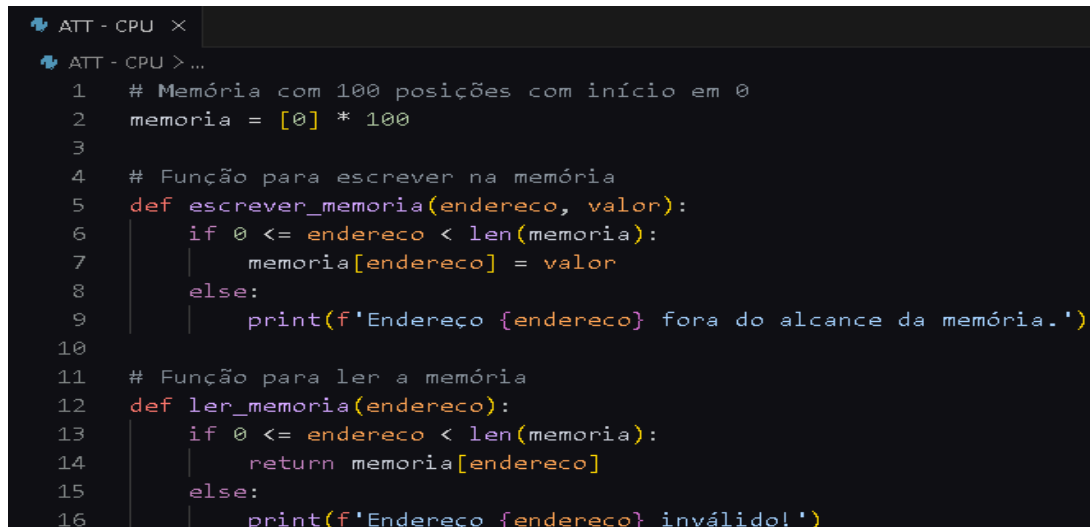
Nesta seção, nosso objetivo é mostrar na prática como funciona um sistema baseado na Arquitetura de Von Neumann. Para isso, a linguagem de programação escolhida foi o Python por ser simples e fácil de entender, principalmente no momento de representar as estruturas lógicas.

Além disso, o Python facilita o trabalho com dados, o que nos auxilia na hora de simular os componentes do computador sem precisar de um hardware real. Um exemplo que será mostrado a seguir é o da Memória Principal (RAM), que pode ser representada por uma **list** (Lista), os registradores por **variáveis** e a **UC** (Unidade de Controle) por comandos condicionais que irão executar as instruções.

Desse modo, essa simulação servirá para mostrar de uma forma básica como é o funcionamento de uma CPU, partindo da leitura das instruções até a execução, através das estampas de **Buscar** (Fetch), **Decodificar** (Decode) e **Executar** (Execute).

2.2.1 - Memória e Interface de Acesso

A Memória Principal ou RAM é um espaço único e compartilhado, nele as instruções do programa e os dados podem existir juntas. Na nossa simulação, a memória é representada por uma lista com 100 posições, onde cada uma é inicializada com 0, desse modo é possível garantir neutralidade para o hardware antes de carregar com um software, abaixo está a **Figura 1 - Memória de Funções**.



```

ATT - CPU > ...
1  # Memória com 100 posições com início em 0
2  memoria = [0] * 100
3
4  # Função para escrever na memória
5  def escrever_memoria(endereco, valor):
6      if 0 <= endereco < len(memoria):
7          memoria[endereco] = valor
8      else:
9          print(f'Endereço {endereco} fora do alcance da memória.')
10
11 # Função para ler a memória
12 def ler_memoria(endereco):
13     if 0 <= endereco < len(memoria):
14         return memoria[endereco]
15     else:
16         print(f'Endereço {endereco} inválido!')

```

Como observado na Figura 1, foi criada a base do sistema onde ocorre o armazenamento. A variável `memória` (linha 2) define o limite do sistema (100). Já na parte das funções, `escrever_memoria` (linha 5) e `ler_memoria` (linha 12) funcionam como **barramentos de controle de dados**. Foi criada também uma camada de segurança (linhas 6 e 13) que validam se o endereço que foi solicitado está dentro do alcance permitido de 0 a 99, como se fosse uma proteção de memória que impede falhas de segmentação no hardware

2.2.2 - CPU e Registradores

A CPU é o componente responsável pelo processo das instruções buscadas na memória. Em nosso simulador, foram criadas duas variáveis que representam os registradores internos, que armazenam dados temporários e o estado atual da execução.

- **Acumulador (ACC):** É o registrador principal, utilizado com a finalidade de armazenar os resultados das operações da Unidade de Lógica e Aritmética (ULA).
- **Program Counter (PC):** É o registrador que armazena o endereço da próxima instrução que será buscada e executada na memória.

Figura 2 - Registradores

```
18  # Registradores (CPU)
19  ACC = 0 # Acumulador
20  PC = 0 # Program Counter
```

Ao analisar a imagem 2, vimos que foram criadas duas variáveis que são iniciadas com o valor zero. O **PC** atua como um direcionador para a lista da memória; a cada novo ciclo de instrução, seu valor é alterado para que o sistema consiga ler qual posição deve ler.

2.2.3 - Conjunto de Instruções (ISA) e Opcodes

O Instruction Set Architecture ou ISA é quem define as operações que o hardware irá realizar. Cada instrução deve ser representada por um código numérico chamado **Opcode**. Na nossa simulação, escolhi adotar uma instrução de largura fixa, ou seja, cada comando será composto por um par: o **código da operação** e o seu **dado/endereço de memória**. Dessa forma, o comando irá percorrer as memória de 2 em 2 (0-1, ..., 98-99).

Figura 3 - ISA

```
22  # CONJUNTO DE INSTRUÇÕES (ISA)
23  # 10: LOAD - Carrega um valor da memória para o ACC
24  # 20: ADD - Soma o valor da memória ao ACC atual
25  # 30: STORE - Armazena o valor do ACC em um endereço da memória
26  # 0: HALT - Para a execução do programa
```

A figura 3 mostra a lista de instruções que podem ser usadas e seus respectivos **códigos numéricos** (opcode). O opcode **10 (LOAD)** é responsável por mover os dados da memória para o ACC, o **20 (ADD)** adiciona a Unidade Lógica e Aritmética para realizar somas ao ACC, o **30 (STORE)** escreve o valor atual do ACC na memória RAM e o **0 (HALT)** é quem executa um break e força o sistema parar. Desse modo, é evidente que a padronização numérica dos **Opcodes** é importante

para que a Unidade de Controle consiga decodificar e direcionar o fluxo de dados para o caminho correto.

2.2.4 - Entrada de dados e Tratamento de Exceções

Nesta seção iremos definir a interatividade do sistema com a implementação de um código que permita o usuário inserir os Opcodes e os Dados diretamente na memória RAM simulada. Mas como o abrimos possibilidade para que o usuário defina esses valores, também devemos pensar em possíveis interrupções por falhas humanas, como strings ou números não inteiros, sendo criado então uma camada de segurança baseada no tratamento de exceções.

Figura 4 - Sistema interativo e camada de segurança

```
28 # Entrada do opcode e do dado
29 while proximo_end < 100:
30     try:
31         op = input(f'Endereço [{proximo_end}] - Opcode: ')
32         opcode = int(op)
33         escrever_memoria(proximo_end, opcode)
34
35         if opcode == 0: break # HALT
36
37         proximo_end += 1 # Carrega o Dado na próxima posição
38         dado = int(input(f'Endereço [{proximo_end}] - Dado: '))
39         escrever_memoria(proximo_end, dado)
40         proximo_end += 1 # Avança para o próximo par
41
42     except ValueError:
43         print('Entrada inválida! Digite apenas números.')
44
```

Ao analisar a figura 4, vemos que iniciamos com a estrutura **while** e o bloco **try/except**. O **try** é um comando que permite o sistema processar a entrada do teclado, nesse caso, se o usuário digitou um dado não numérico, o **except ValueError** impede que o programa seja encerrado, capturando o erro e emitindo um alerta com o **print**. Além disso, temos a lógica de acionamento da variável **proximo_end** de 2

em 2, garantindo a organização linear da memória, respeitando o padrão criado (Instrução e Dado) que definimos anteriormente na arquitetura.

2.2.5 - Unidade de Processamento (UC) e Ciclo de Instrução (Fetch-Decode-Execute)

Após passar os dados do programa para a memória, o nosso simulador começa a fase do processamento real. Essa fase é conhecida como o ciclo de instruções: **Busca, Decodificação e Execução** (Fetch-Decode-Execute). Para garantir que esse processo seja automático, foi criado um laço de repetição com **while True**, que simula o sinal de clock do processador, mantendo a CPU ligada até que a instrução de parada (HALT) seja identificada.

Figura 5 - Ciclo de Instrução

```
45 # Execução do Programa
46 while True:
47     # FETCH - Busca a instrução no endereço que PC indicar
48     instrucao = ler_memoria(PC)
49
50     match instrucao:
51         case 10: # LOAD - Pega o valor do proximo_end para o ACC
52             ACC = ler_memoria(PC + 1) # Lê proximo_end e guarda em ACC
53             print(f'Executando LOAD: ACC = {ACC}')
54             PC += 2 # PC += 2 garante que o ciclo de instrução lera Opcode e Dado
55
56         case 20: # ADD - Soma o proximo_end com o ACC
57             valor_soma = ler_memoria(PC + 1) # Lê o valor a ser somado
58             ACC += valor_soma # Atualiza o ACC com a soma
59             print(f'Executando ADD: Somando {valor_soma}, ACC = {ACC}')
60             PC += 2
61
62         case 30: # STORE - Salva o novo ACC no endereço do PC + 1
63             endereco_destino = ler_memoria(PC + 1) # Vê em qual endereço o resultado será salvo
64             escrever_memoria(endereco_destino, ACC) # Guarda ACC no endereço escolhido
65             print(f'Executando STORE: Salvando {ACC} no endereço {endereco_destino}')
66             PC += 2
67
68         case 0: # HALT - Para a execução do programa
69             print('Execução finalizada via Opcode 0 (HALT)')
70             break
71
72         case _: # Segurança caso Opcode não seja definido
73             print(f'Erro: Instrução inválida {instrucao} no PC {PC}')
74             break
```

A figura 5 mostra o “núcleo” de processamento do simulador funcionando. A estrutura **match case** representa o decodificador da **Unidade de Controle**, ou seja, quem decodifica os sinais elétricos para saber qual instrução usar. Nas linhas 52 e 57 (**LOAD e ADD**), o sistema pega a memória na posição **PC + 1** e extrai os operandos que estão interagindo com o nosso **Acumulador (ACC)**. Já nas linhas 63 e 64 (**STORE**), o valor de **PC + 1** não é um número solto mais, agora é um endereço de destino para salvar na memória RAM. Os **prints** em cada instrução servem para mostrar um pequeno feedback visual em tempo real de como o hardware está usando os dados antes de adicionar ao **Program Counter (PC)** à próxima etapa.

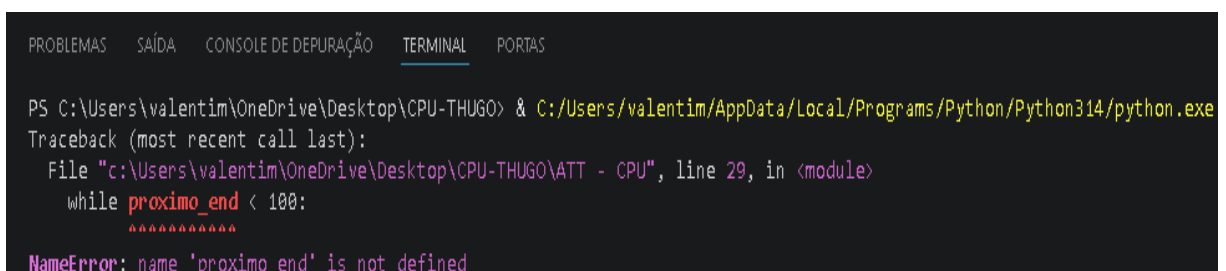
3 - RESULTADOS E TESTES DE EXECUÇÃO

Nessa seção daremos início ao comportamento do simulador durante sua execução. Entretanto, ao realizar o primeiro teste obtive um erro que mostrou a importância de criar variáveis, que na arquitetura de computadores é fundamental para evitar possíveis funções indefinidas no hardware.

3.1 - Testes e ajustes

Antes de iniciar o primeiro teste, o sistema apresentou uma falha de referência de variável no controle do endereçamento no loop responsável pela entrada dos opcodes e dos dados.

Figura 6 - Primeiro Teste

A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMAS', 'SAÍDA', 'CONSOLE DE DEPUÇÃO', 'TERMINAL' (which is selected), and 'PORTAS'. The terminal shows a command prompt 'PS C:\Users\valentim\OneDrive\Desktop\CPU-THUGO>' followed by a command to run a Python script. Below the command, it shows a traceback for a 'NameError' with the message 'name 'proximo_end' is not defined'. The error occurred in a file at 'c:\Users\valentim\OneDrive\Desktop\CPU-THUGO\ATT - CPU', line 29, within a 'while' loop that checks 'proximo_end < 100'.

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPUÇÃO  TERMINAL  PORTAS

PS C:\Users\valentim\OneDrive\Desktop\CPU-THUGO> & C:/Users/valentim/AppData/Local/Programs/Python/Python314/python.exe
Traceback (most recent call last):
  File "c:\Users\valentim\OneDrive\Desktop\CPU-THUGO\ATT - CPU", line 29, in <module>
    while proximo_end < 100:
          ^^^^^^^^^^^
NameError: name 'proximo_end' is not defined
```

A figura 6 mostrou o erro de execução (Traceback) causado pela falta da variável **proximo_end**. Isso nos mostrou que o loop da linha 29 tentou acessar um recurso da memória que ainda não havia sido alocado.

Para resolver esse problema, foi adicionado apenas uma linha (**proximo_end = 0**) antes do loop no código, onde o sistema começou a reconhecer o ponto inicial da memória RAM, permitindo que o usuário insira o primeiro **Opcode** no **endereço zero**, como era esperado na arquitetura.

Figura 7 - Segundo Teste

```
28 # Entrada do opcode e dos dados
29 proximo_end = 0
30 while proximo_end < 100:
```

Após o ajuste final, foi possível continuar para os resultados, pois o sistema passou a reconhecer o ponto inicial da RAM.

3.2 - Simulação de uma Operação Aritmética

Após as devidas correções no sistema, realizamos um teste prático de soma. O objetivo do teste foi carregar um valor na CPU, somar uma segunda unidade e armazenar o resultado em um endereço específico na memória RAM, se assemelhando a um processo completo.

Figura 8 - Load



```
PROBLEMAS  SAÍDA  CONSOLE DE DEPUÇÃO  TERMINAL  PORTAS
PS C:\Users\valentim\OneDrive\Desktop\CPU-THUGO> & C:/Users/valentim/AppData/Local/Programs/Python/Python314/python.exe
Endereço [0] - Opcode: 10
Endereço [1] - Dado: 30
```

Na figura 8, definimos no endereço 0 o **Opcode 10** (LOAD), em seguida no endereço 1 o **dado 30** que foi armazenado no **Acumulador** (ACC).

Figura 9 - Add

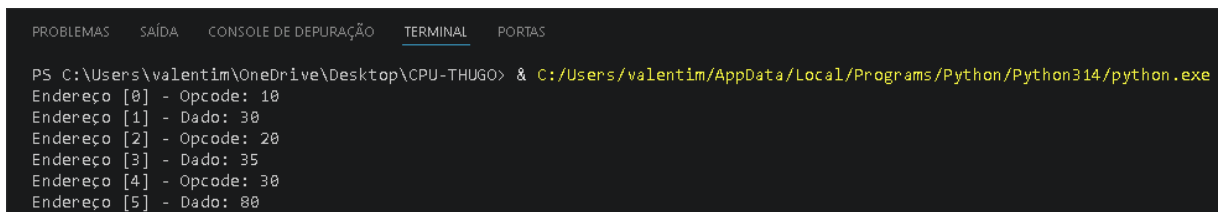


```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURACÃO  TERMINAL  PORTAS

PS C:\Users\valentim\OneDrive\Desktop\CPU-THUGO> & C:/Users/valentim/AppData/Local/Programs/Python/Python314/python.exe
Endereço [0] - Opcode: 10
Endereço [1] - Dado: 30
Endereço [2] - Opcode: 20
Endereço [3] - Dado: 35
```

Em seguida, na figura 9 vemos que no endereço 2 selecionamos o **Opcode 20** (ADD), que recebeu o **dado 35** no endereço 3 para realizar a soma ao valor já armazenado no **ACC** ($ACC = 30 + 35$).

Figura 10 - Store

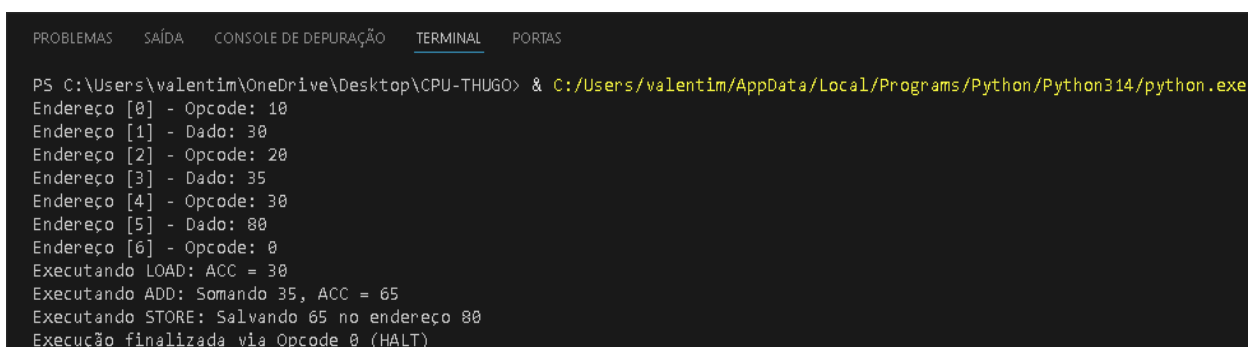


```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURACÃO  TERMINAL  PORTAS

PS C:\Users\valentim\OneDrive\Desktop\CPU-THUGO> & C:/Users/valentim/AppData/Local/Programs/Python/Python314/python.exe
Endereço [0] - Opcode: 10
Endereço [1] - Dado: 30
Endereço [2] - Opcode: 20
Endereço [3] - Dado: 35
Endereço [4] - Opcode: 30
Endereço [5] - Dado: 80
```

Na figura 10, usamos o **Opcode 30** (STORE) no endereço 4 para guardar o resultado das últimas instruções (ADD e LOAD) no **endereço 80**, definido no endereço 5. Nesse momento nosso **Acumulador** deverá valer 65, mediante as instruções: **LOAD = 30** e o **ADD = 35**, logo **ACC = 65**.

Figura 11- Halt



```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURACÃO  TERMINAL  PORTAS

PS C:\Users\valentim\OneDrive\Desktop\CPU-THUGO> & C:/Users/valentim/AppData/Local/Programs/Python/Python314/python.exe
Endereço [0] - Opcode: 10
Endereço [1] - Dado: 30
Endereço [2] - Opcode: 20
Endereço [3] - Dado: 35
Endereço [4] - Opcode: 30
Endereço [5] - Dado: 80
Endereço [6] - Opcode: 0
Executando LOAD: ACC = 30
Executando ADD: Somando 35, ACC = 65
Executando STORE: Salvando 65 no endereço 80
Execução finalizada via Opcode 0 (HALT)
```

Por fim, na figura 11 vemos que no endereço 6 foi selecionado o **Opcode 0** (HALT) que finaliza a carga e a execução do ciclo de instruções. No terminal mostra a execução do **LOAD, ADD, STORE** e por fim sua finalização com **HALT**, mostrando o dado 65 no endereço 80, validando assim a interação entre entrada e saída e o ciclo de instruções do nosso simulador.

4 - CONCLUSÃO

Este projeto permitiu entender de forma prática como a **Arquitetura de Von Neumann** organiza o fluxo de dados no computador. Com esse simulador em Python, ficou claro que a CPU não tem vontade própria, mas sim uma lógica rigorosa com base no Ciclo de Instruções: **Busca, Decodificação e Execução**.

Um dos pontos interessantes foi entender a importância do **Program Counter** (PC) e como ele serve de guia fundamental na máquina, permitindo que ela consiga diferenciar o que é uma instrução e o que é um dado. Também foi criado uma camada de segurança para tratar possíveis exceções, mostrando que um hardware forte depende de como o software pode lidar com falhas humanas e entradas inválidas.

Resumindo, esse projeto mostrou que os conceitos teóricos não são tão complexos como aparentam ser. Ver o **Acumulador** (ACC) recebendo as instruções que eu mesmo digitei no terminal provou que no fim, a computação é uma construção extremamente lógica de passos simples, mas que no fim, ao serem somadas no coletivo, podem realizar tarefas extensas e interessantes.

O código-fonte completo, mostrando todas as funções e a lógica apresentada, se encontra no **Apêndice A** do documento.

5 - BIBLIOGRAFIA

- FÁVERO, E. M. B. **Organização e Arquitetura de Computadores**. Cuiabá: Rede e-Tec Brasil, 2011. Disponível em: <http://proedu.rnp.br>. Acesso em: 08 abr. 2026.
- TANENBAUM, Andrew S.; AUSTIN, Todd. **Arquitetura Estruturada de Computadores**. 6. ed. São Paulo: Pearson Education do Brasil, 2013.

5.1 - Ferramentas e Tecnologias

- **Linguagem Python (3.10)**: Escolhida por sua sintaxe clara e suporte às estruturas escolhidas, como a **match case**, que facilitou a simulação de uma Unidade de Controle.
- **Visual Studio Code (VS Code)**: Ambiente de Desenvolvimento Integrado (IDE) utilizado para a escrita e organização das depurações do código, além de extensões que ajudaram na análise estática e execução dos scripts.

6 - APÊNDICE A

```
# Memória com 100 posições com início em 0

memoria = [0] * 100

# Função para escrever na memória

def escrever_memoria(endereco, valor):

    if 0 <= endereco < len(memoria):

        memoria[endereco] = valor

    else:
```

```
        print(f'Endereço {endereco} fora do alcance
da memória.')

# Função para ler a memória

def ler_memoria(endereco):

    if 0 <= endereco < len(memoria):

        return memoria[endereco]

    else:

        print(f'Endereço {endereco} inválido!')

# Registradores (CPU)

ACC = 0 # Acumulador

PC = 0 # Program Counter

# CONJUNTO DE INSTRUÇÕES (ISA)

# 10: LOAD - Carrega um valor da memória para o ACC

# 20: ADD - Soma o valor da memória ao ACC atual

# 30: STORE - Armazena o valor do ACC em um endereço
da memória

# 0: HALT - Para a execução do programa

# Entrada do opcode e dos dados

proximo_end = 0

while proximo_end < 100:
```

```

        try:

            op = input(f'Endereço [{proximo_end}] -
Opcode: ')

            opcode = int(op)

            escrever_memoria(proximo_end, opcode)

            if opcode == 0: break # HALT

            proximo_end += 1 # Carrega o Dado na próxima
posição

            dado = int(input(f'Endereço [{proximo_end}] -
Dado: '))

            escrever_memoria(proximo_end, dado)

            proximo_end += 1 # Avança para o próximo par

        except ValueError:

            print('Entrada inválida! Digite apenas
números.')

# Execução do Programa

while True:

    # FETCH - Busca a instrução no endereço que PC
indica

    instrucao = ler_memoria(PC)

```



```

match instrucao:

    case 10: # LOAD - Pega o valor do proximo_end
para o ACC

        ACC = ler_memoria(PC + 1) # Lê
proximo_end e guarda em ACC

        print(f'Executando LOAD: ACC = {ACC}')

        PC += 2 # PC += 2 garante que o ciclo de
instrução lera Opcode e Dado


    case 20: # ADD - Soma o proximo_end com o ACC

        valor_soma = ler_memoria(PC + 1) # Lê o
valor a ser somado

        ACC += valor_soma # Atualiza o ACC com a
soma

        print(f'Executando ADD: Somando
{valor_soma}, ACC = {ACC}')

        PC += 2


    case 30: # STORE - Salva o novo ACC no
endereço do PC + 1

        endereco_destino = ler_memoria(PC + 1) #
Vê em qual endereço o resultado será salvo

        escrever_memoria(endereco_destino, ACC) #
Guarda ACC no endereço escolhido

        print(f'Executando STORE: Salvando {ACC}
no endereço {endereco_destino}')

        PC += 2

```

```
        case 0: # HALT - Para a execução do programa

            print('Execução finalizada via Opcode 0
(HALT) ')

            break

        case _: # Segurança caso Opcode nao seja
definido

            print(f'Erro: Instrução inválida
{instrucao} no PC {PC} ')

            break
```